

Table of contents

Série 1	1
Partie 1 : Méthodes de Résolution	1
Méthodes pour les Tautologies et Validité	1
Méthodes pour la Logique du Premier Ordre	2
Méthodes pour les Problèmes de Satisfaction	2
Rappel des Lois Logiques Fondamentales	3
Lois de l'Implication	3
Lois de De Morgan	3
Lois Distributives	3
Lois d'Associativité et Commutativité	3
Lois d'Identité et Complément	3
Partie 2 : Exemples Illustratifs	4
Exemple 1 : Transformation par Réécriture (Modus Ponens)	4
Exemple 2 : Recherche de Contre-Exemple	4
Exemple 3 : Formalisation en Logique du Premier Ordre	5
Exemple 4 : Modélisation de Contraintes d'Unicité	5
Exemple 5 : Raisonnement Systématique (Problème des Boîtes)	6
Exemple 6 : Forme Normale Conjonctive	6
Diagramme des Méthodes	7
Synthèse	8

Série 1

Partie 1 : Méthodes de Résolution

Méthodes pour les Tautologies et Validité

1. Table de vérité complète

Méthode exhaustive qui évalue toutes les combinaisons possibles des variables pour déterminer si une formule est toujours vraie.

2. Transformation par réécriture

Utilisation systématique des lois logiques pour simplifier ou transformer les formules jusqu'à obtenir une forme évidente.

3. Recherche de contre-exemple

Pour prouver qu'une formule n'est **pas** une tautologie, on cherche une assignation des variables qui rend la formule fausse.

4. Forme normale conjonctive (CNF)

Transformation d'une formule en conjonction de disjonctions de littéraux, permettant de vérifier facilement la validité.

5. Analyse par cas

Examen systématique de tous les scénarios possibles pour une variable ou une sous-formule.

Méthodes pour la Logique du Premier Ordre

6. Identification des prédicats et constantes

Définition précise des prédicats, fonctions et constantes nécessaires pour modéliser un énoncé en langage naturel.

7. Traduction guidée par la structure logique

Utilisation des connecteurs logiques ($\forall$, $\exists$, $\Rightarrow$, $\wedge$, $\vee$, $\neg$) pour refléter la structure de l'énoncé.

8. Gestion des quantificateurs

Placement correct des quantificateurs pour capturer la portée des variables.

9. Modélisation de contraintes d'unicité

Utilisation de combinaisons de quantificateurs et d'égalité pour exprimer l'unicité.

Méthodes pour les Problèmes de Satisfaction

10. Formalisation propositionnelle

Transformation d'un problème concret en variables propositionnelles et contraintes logiques.

11. Satisfiabilité par raisonnement systématique

Examen des contraintes et recherche d'assignations cohérentes par élimination des cas impossibles.

Rappel des Lois Logiques Fondamentales

Lois de l'Implication

- Élimination de l'implication : $p \Rightarrow q \equiv \neg p \vee q$
- Contraposée : $p \Rightarrow q \equiv \neg q \Rightarrow \neg p$
- Négation d'une implication : $\neg(p \Rightarrow q) \equiv p \wedge \neg q$

Lois de De Morgan

- $\neg(p \wedge q) \equiv \neg p \vee \neg q$
- $\neg(p \vee q) \equiv \neg p \wedge \neg q$

Lois Distributives

- $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
- $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$

Lois d'Associativité et Commutativité

- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
- $(p \vee q) \vee r \equiv p \vee (q \vee r)$
- $p \wedge q \equiv q \wedge p$
- $p \vee q \equiv q \vee p$

Lois d'Identité et Complément

- $p \vee \text{False} \equiv p$
 - $p \wedge \text{True} \equiv p$
 - $p \vee \neg p \equiv \text{True}$
 - $p \wedge \neg p \equiv \text{False}$
-

Partie 2 : Exemples Illustratifs

Exemple 1 : Transformation par Réécriture (Modus Ponens)

Formule : $(p \wedge (p \Rightarrow q)) \Rightarrow q$

Application des méthodes :

- Utilisation de l'élimination de l'implication
- Application de De Morgan
- Utilisation de la distributivité
- Simplification par les lois du complément

Démonstration :

$$\begin{aligned} & (p \wedge (p \Rightarrow q)) \Rightarrow q \\ \equiv & \neg(p \wedge (p \Rightarrow q)) \vee q \quad (\text{élimination de } \Rightarrow) \\ \equiv & \neg(p \wedge (\neg p \vee q)) \vee q \quad (\text{élimination de } \Rightarrow) \\ \equiv & (\neg p \vee \neg(\neg p \vee q)) \vee q \quad (\text{De Morgan}) \\ \equiv & (\neg p \vee (p \wedge \neg q)) \vee q \quad (\text{De Morgan}) \\ \equiv & ((\neg p \vee p) \wedge (\neg p \vee \neg q)) \vee q \quad (\text{distributivité}) \\ \equiv & (\text{True} \wedge (\neg p \vee \neg q)) \vee q \quad (\text{complément}) \\ \equiv & (\neg p \vee \neg q) \vee q \\ \equiv & \neg p \vee (\neg q \vee q) \\ \equiv & \neg p \vee \text{True} \\ \equiv & \text{True} \end{aligned}$$

Exemple 2 : Recherche de Contre-Exemple

Formule : $(q \vee r \Rightarrow p \wedge \neg s) \wedge (s \vee p \Rightarrow (\neg r \Rightarrow q)) \Rightarrow \neg(s \vee r)$

Application des méthodes :

- Analyse de la structure d'implication
- Recherche d'assignation rendant prémisse vraie et conclusion fausse
- Vérification de cohérence

Démonstration :

1. **Pour que la conclusion soit fausse :** $\neg(s \vee r)$ doit être fausse, donc $s \vee r$ doit être vraie. Choix : $s = \text{False}$, $r = \text{True}$

2. Pour que la prémisse soit vraie :

- Première partie : $q \vee r \Rightarrow p \wedge \neg s$ Avec $r = \text{True}$, $q \vee r$ est vraie. Pour que l'implication soit vraie, $p \wedge \neg s$ doit être vraie. Avec $s = \text{False}$, $\neg s$ est vraie, donc p doit être vraie.
- Deuxième partie : $s \vee p \Rightarrow (\neg r \Rightarrow q)$ Avec $p = \text{True}$, $s \vee p$ est vraie. $\neg r$ est faux (car r vrai), donc $\neg r \Rightarrow q$ est vraie (prémisse fausse).

3. **Assignment trouvée** : $p = \text{True}$, q quelconque, $r = \text{True}$, $s = \text{False}$ Cette assignation rend la prémisse vraie et la conclusion fausse, donc la formule n'est pas une tautologie.

Exemple 3 : Formalisation en Logique du Premier Ordre

Énoncé : “Toute personne qui travaille en informatique a écrit au moins un programme”

Application des méthodes :

- Identification des prédicats : $\text{CS}(x)$, $\text{P}(y)$, $W(x, y)$
- Analyse de la structure : “pour tout x , si $\text{CS}(x)$ alors il existe y tel que...”
- Traduction guidée par la structure logique

Formalisation :

$$\forall x (\text{CS}(x) \Rightarrow \exists y (\text{P}(y) \wedge W(x, y)))$$

Exemple 4 : Modélisation de Contraintes d'Unicité

Énoncé : “Chaque programme a été écrit par une unique personne qui travaille en informatique”

Application des méthodes :

- Expression de l'existence : $\exists y$
- Expression de l'unicité : $\forall z ((\text{CS}(z) \wedge W(z, x)) \Rightarrow z = y)$
- Combinaison avec conjonction

Formalisation :

$$\forall x (\text{P}(x) \Rightarrow \exists y (\text{CS}(y) \wedge W(y, x) \wedge \forall z ((\text{CS}(z) \wedge W(z, x)) \Rightarrow z = y)))$$

Exemple 5 : Raisonnement Systématique (Problème des Boîtes)

Problème : Trouver quelle boîte contient le trésor avec les affirmations contradictoires

Application des méthodes :

1. **Formalisation propositionnelle :**

- Variables T_i : trésor dans la boîte i
- Variables B_i : affirmation sur la boîte i est vraie

2. **Traduction des contraintes :**

- $B_1 \Leftrightarrow \neg T_1$
- $B_2 \Leftrightarrow \neg T_2$
- $B_3 \Leftrightarrow T_1$
- Exactement une B_i vraie
- Exactement une T_i vraie

3. **Analyse par cas :**

- Cas 1 : T_1 vraie $\rightarrow$ contradiction avec B_2
- Cas 2 : T_3 vraie $\rightarrow$ contradiction avec unicité des B_i
- Cas 3 : T_2 vraie $\rightarrow$ toutes les contraintes satisfaites

Conclusion : Le trésor est dans B_2 .

Exemple 6 : Forme Normale Conjonctive

Formule : $(\neg q \Rightarrow \neg p) \Rightarrow ((\neg q \Rightarrow p) \Rightarrow q)$

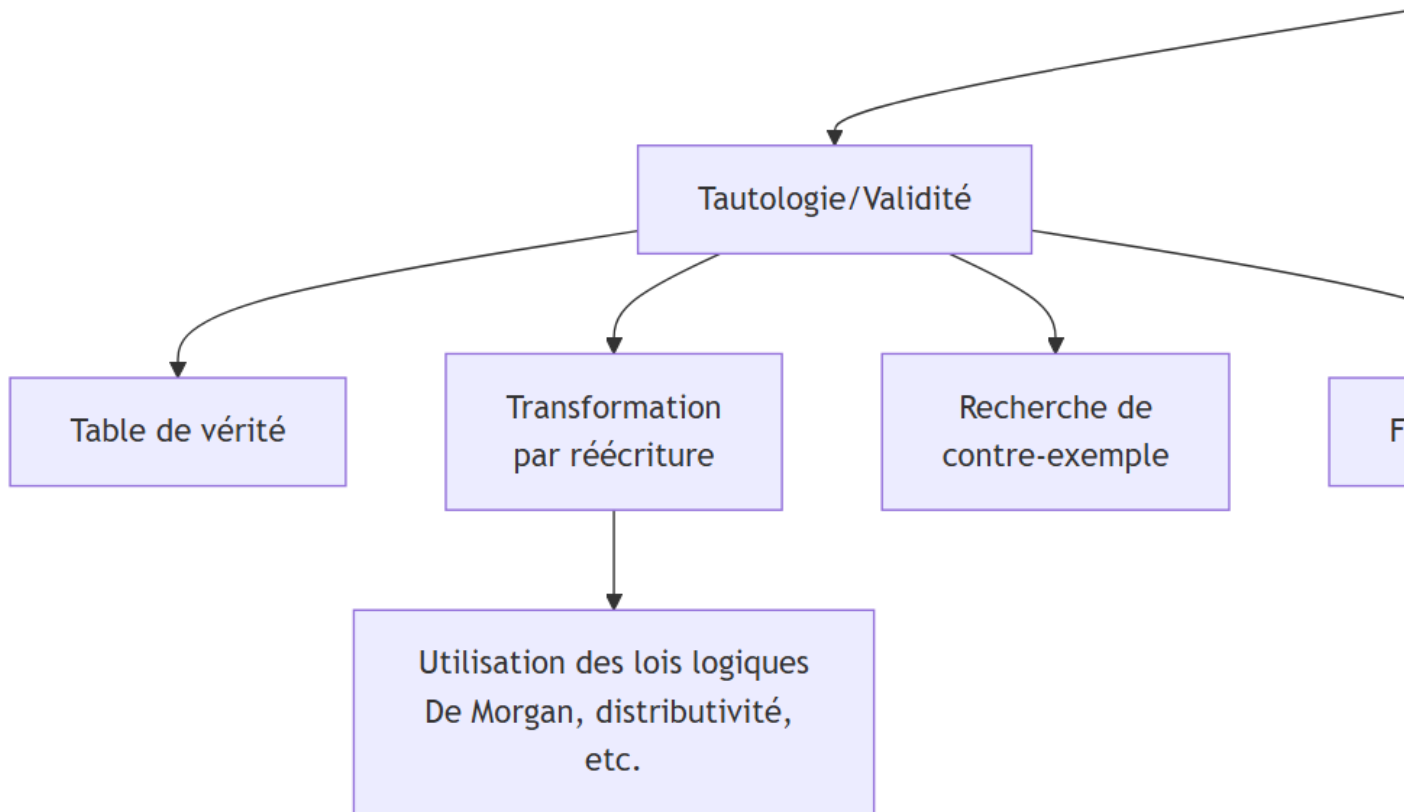
Application des méthodes :

- Transformation en CNF
- Observation que chaque clause contient un littéral et sa négation
- Conclusion de validité

Démonstration (extrait) :

Après transformations, on obtient une CNF où chaque clause est de la forme $(p \vee q \vee \neg q)$ ou similaire, donc toujours vraie.

Diagramme des Méthodes



Synthèse

Les méthodes présentées couvrent les principaux aspects de la résolution de problèmes en logique propositionnelle et du premier ordre. La maîtrise de ces méthodes, combinée à une bonne connaissance des lois logiques fondamentales, permet d'aborder une large variété de problèmes en modélisation et vérification de logiciels.

Points clés :

1. Toujours commencer par comprendre la structure logique du problème
2. Choisir la méthode adaptée au type de problème (validation, modélisation, satisfaction)
3. Appliquer systématiquement les lois logiques dans les transformations
4. Vérifier la cohérence des solutions trouvées
5. Documenter clairement chaque étape du raisonnement